



School of Computer Science and Engineering

Winter Semester 2022-2023

Continuous Assessment Test – 1

SLOT: A2+TA2

Programme Name & Branch: B Tech Computer Science & Engineering

Course Name & code: BCSE202L Data Structures and Algorithms

Class Number (s): VL2022230506314, 5850, 5856, 5839, 5843, 5858, 5841, 6333, 5848

Faculty Name (s): S M Farooq, Ganesh S K, Keerthika P, Sayan S, Jafar Ali Ibrahim, Sunil Kumar, Lakshmanan K, Kalaivani K, Priyanka N, Thurai Pandian M

Exam Duration: 90 Min.

Maximum Marks: 50

General instruction(s):

Answer all questions.

Q.No	Question	Max Marks	CO	BL										
1.	Generate polynomial functions for the following three code segments and compute the time complexity. (3+3+4) Segment-1 <pre>test1() { int j = 0, k = 2; while(j > 0) { j += k; k *= 2; } }</pre> Ans:- Constant complexity, as the loop would never get executed. Segment-2 <pre>int test2(int n) { int k = 0; for (int x = 0; x < n; x++) for (int y = n; y > x; y--) k += x + y; return (k); }</pre> Ans:- <table><tr><th>n</th><th># times inner loop is invoked</th></tr><tr><td>1</td><td>1</td></tr><tr><td>2</td><td>3</td></tr><tr><td>3</td><td>6</td></tr><tr><td>4</td><td>10</td></tr></table>	n	# times inner loop is invoked	1	1	2	3	3	6	4	10	10	CO 1	BL 3
n	# times inner loop is invoked													
1	1													
2	3													
3	6													
4	10													

	<table><tr><td>5</td><td>15</td></tr><tr><td>6</td><td>21</td></tr><tr><td>7</td><td>28</td></tr></table> This follows the pattern that: $T_0 = 0$ $T_1 = T_0 + 1$ $T_2 = T_1 + 2$ $T_3 = T_2 + 3$ $\dots$ $T_n = T_{n-1} + n$ After Summing we get: $T_n = T_0 + 1 + 2 + 3 + \dots + n = T_0 + \frac{n(n+1)}{2} = O(n^2)$	5	15	6	21	7	28			
5	15									
6	21									
7	28									
	Segment-3 <pre>int test3(int n) { int x, y, z = 0; for (x = n/2; x <= n; x++) for (y=2; y<=n; y *= 2) z += n/2; return (z); }</pre>									
	Ans:- The inner loop would work for $\log n$ times and outer would for $\frac{n}{2}$ times for any n. Therefore, $T(n) = O(\frac{n}{2} \log n) = O(n \log n)$									
2.	a) Solve the following recurrence using recurrence tree method (5) $T(n) = T(n/5) + T(4n/5) + n$ Ans:- Keeping the lower term on the left subtree and higher term on the right subtree, the best case would be on the leftmost path and worst case would be on the rightmost path from root. Considering the rightmost path for a worst-case analysis. substituting $\frac{n}{(\frac{5}{4})^k} = 1 \Rightarrow k = \log_{\frac{5}{4}} n$ Sum at each level = n $\therefore T(n) = n + n + \dots + n$ (k times), but $k = \log_{\frac{5}{4}} n$ $\Rightarrow T(n) = n \log_{\frac{5}{4}} n$	10	CO 1	BL 3						
	b) Solve the following recurrences using Master's method (2+3) i) $T(n) = 2T(n-1) + n$ Ans:- Applying Master's theorem for decreasing functions, $T(n) = O(n2^n)$ ii) $T(n) = 4T(\frac{n}{2}) + n^3/\log^2 n$									

	Ans:- Applying Master's theorem for dividing functions, $T(n) = O(n^3)$																																																																		
3.	a) In a stack, four numbers 1, 2, 3, 4 can be pushed or popped in any random order. Each number can be pushed or popped multiple times. Popping a number prints it. The sequence of operations performed on the stack is Push $\rightarrow$ Push $\rightarrow$ Pop $\rightarrow$ Push $\rightarrow$ Pop $\rightarrow$ Pop $\rightarrow$ Push $\rightarrow$ Pop $\rightarrow$ Push. This sequence printed the numbers 1, 1, 3, 2 in that order. Which number was pushed first? Justify your answer. (5) Ans:- The first number pushed would be 3. Explanation:- 2 Pushes are followed by a Pop that prints 1. Hence, the second number pushed is 1. Then, one push is followed by two pops that print 1, 3. Hence, the third number pushed is 1 and the first number pushed is 3. Hence, 3 is pushed first. b) Discuss the limitation of a linear queue. Illustrate how this is overcome in circular queue with the help of a suitable example. (5) Ans:- Space misutilization. Since the full condition for linear queue is $rear = MAX_SIZE$, even if some elements are dequeued and empty spaces are available in the front, it would say queue is full. So we cannot enqueue, even if there are vacant spaces in the end. Any arbitrary example that shows the working of circular queue would suffice. The enqueue is possible until vacant spaces are available in the queue, as the full condition is $front = (rear + 1) \bmod MAX_SIZE$.	10	CO 2	BL 3																																																															
4.	Convert $(A/B) \wedge C + ((D + E) * F \wedge C)$ to postfix and then evaluate the expression for $A = 18, B = 3, C = 2, D = 13, E = 7$, and $F = 5$ using an appropriate data structure. The processing of each symbol and status of the data structure at each step is expected in both conversion and evaluation. Ans:- $(A/B) \wedge C + ((D + E) * F \wedge C)$ <table><thead><tr><th>Symbol</th><th>Stack</th><th>Expression</th></tr></thead><tbody><tr><td>(</td><td>(</td><td></td></tr><tr><td>A</td><td>(</td><td>A</td></tr><tr><td>/</td><td>(/</td><td>A</td></tr><tr><td>B</td><td>(/</td><td>AB</td></tr><tr><td>)</td><td></td><td>AB/</td></tr><tr><td>^</td><td>^</td><td>AB/</td></tr><tr><td>C</td><td>^</td><td>AB/C</td></tr><tr><td>+</td><td>+</td><td>AB/C^</td></tr><tr><td>(</td><td>+(</td><td>AB/C^</td></tr><tr><td>(</td><td>+((</td><td>AB/C^</td></tr><tr><td>D</td><td>+((</td><td>AB/C^D</td></tr><tr><td>+</td><td>+((+</td><td>AB/C^D</td></tr><tr><td>E</td><td>+((+</td><td>AB/C^DE</td></tr><tr><td>)</td><td>+(</td><td>AB/C^DE+</td></tr><tr><td>*</td><td>+(*</td><td>AB/C^DE+</td></tr><tr><td>F</td><td>+(*</td><td>AB/C^DE+F</td></tr><tr><td>^</td><td>+(* ^</td><td>AB/C^DE+F</td></tr><tr><td>C</td><td>+(* ^</td><td>AB/C^DE+FC</td></tr><tr><td>)</td><td>+</td><td>AB/C^DE+FC^*</td></tr><tr><td></td><td></td><td>AB/C^DE+FC^*+</td></tr></tbody></table>	Symbol	Stack	Expression	(	(		A	(	A	/	(/	A	B	(/	AB	)		AB/	^	^	AB/	C	^	AB/C	+	+	AB/C^	(	+(	AB/C^	(	+((	AB/C^	D	+((	AB/C^D	+	+((+	AB/C^D	E	+((+	AB/C^DE	)	+(	AB/C^DE+	*	+(*	AB/C^DE+	F	+(*	AB/C^DE+F	^	+(* ^	AB/C^DE+F	C	+(* ^	AB/C^DE+FC	)	+	AB/C^DE+FC^*			AB/C^DE+FC^*+	10	CO 2	BL 3
Symbol	Stack	Expression																																																																	
(	(																																																																		
A	(	A																																																																	
/	(/	A																																																																	
B	(/	AB																																																																	
)		AB/																																																																	
^	^	AB/																																																																	
C	^	AB/C																																																																	
+	+	AB/C^																																																																	
(	+(	AB/C^																																																																	
(	+((	AB/C^																																																																	
D	+((	AB/C^D																																																																	
+	+((+	AB/C^D																																																																	
E	+((+	AB/C^DE																																																																	
)	+(	AB/C^DE+																																																																	
*	+(*	AB/C^DE+																																																																	
F	+(*	AB/C^DE+F																																																																	
^	+(* ^	AB/C^DE+F																																																																	
C	+(* ^	AB/C^DE+FC																																																																	
)	+	AB/C^DE+FC^*																																																																	
		AB/C^DE+FC^*+																																																																	

	A = 18; B =3; C = 2; D = 13; E = 7; and F = 5 AB/C^DE+FC^*+ <table><tr><th>Symbol</th><th>Stack</th><th>Evaluation</th></tr><tr><td>A=18</td><td>18</td><td></td></tr><tr><td>B=3</td><td>18 3</td><td></td></tr><tr><td>/</td><td></td><td>18/3 = 6</td></tr><tr><td></td><td>6</td><td></td></tr><tr><td>C=2</td><td>6 2</td><td></td></tr><tr><td>^</td><td></td><td>6^2=36</td></tr><tr><td></td><td>36</td><td></td></tr><tr><td>D=13</td><td>36 13</td><td></td></tr><tr><td>E=7</td><td>36 13 7</td><td></td></tr><tr><td>+</td><td></td><td>13+7 = 20</td></tr><tr><td></td><td>36 20</td><td></td></tr><tr><td>F=5</td><td>36 20 5</td><td></td></tr><tr><td>C=2</td><td>36 20 5 2</td><td></td></tr><tr><td>^</td><td></td><td>5^2=25</td></tr><tr><td></td><td>36 20 25</td><td></td></tr><tr><td>*</td><td></td><td>20*25=500</td></tr><tr><td></td><td>36 500</td><td></td></tr><tr><td>+</td><td></td><td>36+500 = 536</td></tr></table>	Symbol	Stack	Evaluation	A=18	18		B=3	18 3		/		18/3 = 6		6		C=2	6 2		^		6^2=36		36		D=13	36 13		E=7	36 13 7		+		13+7 = 20		36 20		F=5	36 20 5		C=2	36 20 5 2		^		5^2=25		36 20 25		*		20*25=500		36 500		+		36+500 = 536			
Symbol	Stack	Evaluation																																																											
A=18	18																																																												
B=3	18 3																																																												
/		18/3 = 6																																																											
	6																																																												
C=2	6 2																																																												
^		6^2=36																																																											
	36																																																												
D=13	36 13																																																												
E=7	36 13 7																																																												
+		13+7 = 20																																																											
	36 20																																																												
F=5	36 20 5																																																												
C=2	36 20 5 2																																																												
^		5^2=25																																																											
	36 20 25																																																												
*		20*25=500																																																											
	36 500																																																												
+		36+500 = 536																																																											
5.	a) Can you improve the best-case complexity of bubble sort by optimizing the traditional algorithm? If yes, write the modified version of the algorithm and demonstrate how the improvement is achieved. (5) Ans:- Yes. But to achieve this, the traditional bubble sort algorithm must be modified a bit. The optimized pseudocode is shown below: <pre>// An optimized version of Bubble Sort void bubbleSort(int arr[], int n) { int i, j; bool swapped; for (i = 0; i < n-1; i++) { swapped = false; for (j = 0; j < n-i-1; j++) { if (arr[j] > arr[j+1]) { swap(&arr[j], &arr[j+1]); swapped = true; } } // IF no two elements were swapped by inner loop, then break if (swapped == false) break; } }</pre> Explanation:- If no elements are swapped in the first iteration itself, it is a clear indication that the array is already sorted. In that case, the algorithm can terminate. But to check this, one iteration is needed, which takes $O(n)$ comparisons. However, all the remaining iterations/comparisons can be skipped. In this way, the optimized version keeps the complexity $O(n)$ for the best-case. b) Imagine we have two sorted lists with no common elements. Suppose we need to search for an element to see whether it is present in either of the lists. Write an efficient algorithm for this and analyze the time complexity of your algorithm (5) Ans:- Step1: Perform binary search on List1	10	CO 3	BL 4																																																									

	Step2: Perform binary search on List2 Complexity = $O(\log m + \log n) = O(\log n)$ (Assuming n to be the size of the larger list and m is that of the smaller.			
--	--	--	--	--